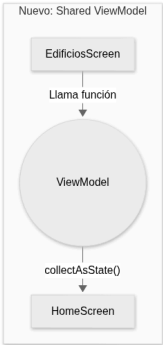
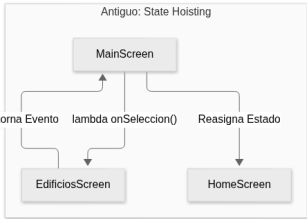


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Introducción al Desarrollo de Nuevas Plataformas				
TÍTULO DE LA PRÁCTICA:	Pantallas múltiples con NavigationBar y comunicación entre Composables en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	05	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	03/10/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Jorge Luis Mamani Huarsaya - Yordano Hernan Boza Portilla				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS DE LOS EJERCICIOS PROPUESTOS
Desarrollo de la Implementación Para cumplir con los ejercicios propuestos en el laboratorio, se realizó una transición desde una arquitectura basada en el pase de funciones lambda (State Hoisting básico) hacia un modelo más maduro y escalable centralizado en el uso de un Shared ViewModel. A continuación, se detalla el proceso de diseño y las justificaciones técnicas detrás de la implementación a lo largo de cinco pasos fundamentales. 1. Diseño del ViewModel Compartido En aplicaciones Compose, cuando múltiples pantallas necesitan acceder o modificar un mismo dato (en este caso, el nombre del edificio seleccionado), pasar este valor de ida y vuelta a través de lambdas produce un acoplamiento fuerte y vuelve el código difícil de mantener (un fenómeno conocido como <i>prop drilling</i>). Para solucionarlo, creamos la clase <code>SeleccionViewModel</code> que hereda de la clase <code>ViewModel</code> base de Android [1]. Dentro de esta clase, se optó por implementar el estado mediante un <code>StateFlow</code> (<code>MutableStateFlow</code>) en lugar del clásico <code>mutableStateOf</code>. La razón técnica para preferir <code>StateFlow</code> es que pertenece de manera nativa a la librería de corrutinas de Kotlin [2], lo que permite un manejo asíncrono puro y facilita enormemente la futura aplicación de pruebas unitarias o la transformación de flujos complejos en un entorno Clean Architecture. Además, para garantizar el principio de encapsulamiento, el estado interno mutable (<code>_edificioSeleccionado</code>) se mantiene privado y se expone públicamente un flujo derivado de solo lectura. Nuevo: Shared ViewModel  Antiguo: State Hoisting  Figura 1: Comparativa de flujos: <i>State Hoisting</i> (lambdas) vs <i>Shared ViewModel</i>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

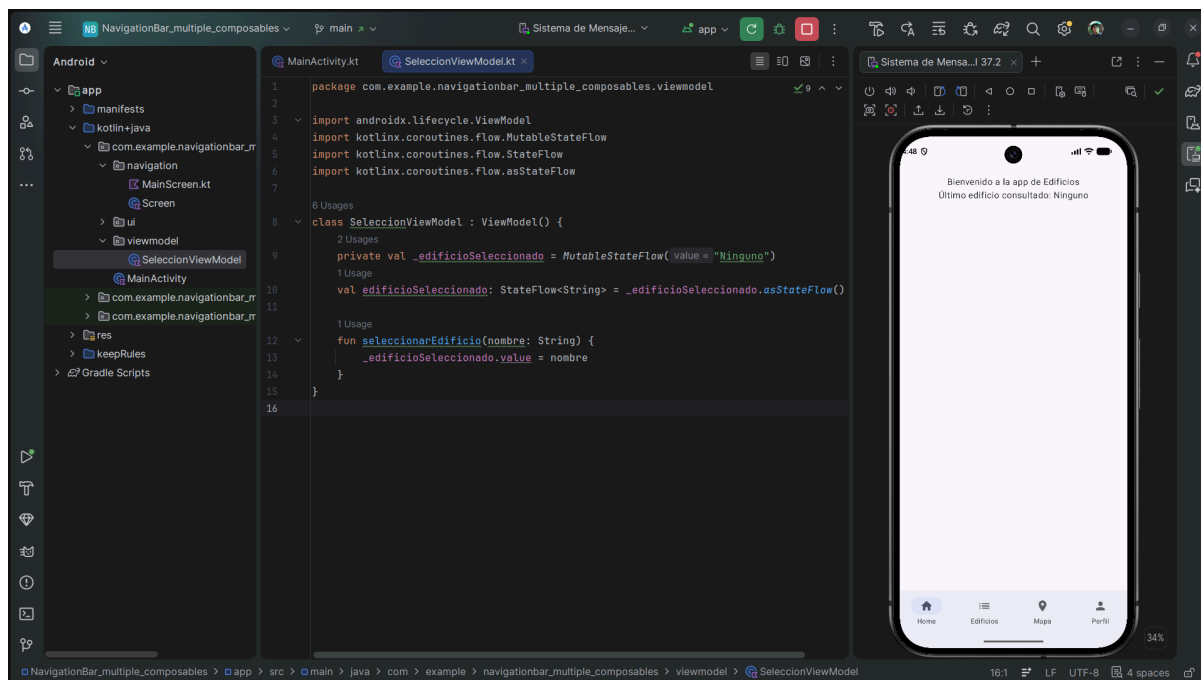


Figura 2: Implementación encapsulada del estado en SeleccionViewModel

2. Inyección de Estado a nivel de NavGraph

El éxito de un **Shared ViewModel** radica en definir correctamente su ámbito de vida (*scope*). Si instanciáramos el ViewModel independientemente dentro de cada pantalla (es decir, creando uno nuevo para Edificios y otro nuevo para Home), obtendríamos dos espacios de memoria completamente distintos y la información no se compartiría.

Para que EdificiosScreen y HomeScreen accedan exactamente al mismo objeto en memoria, la instancia única se solicitó en el nivel jerárquico superior dentro de MainScreen.kt, empleando la función generadora `viewModel()`. Al realizar esto en el Composable padre que contiene al NavHost, la instancia de `SeleccionViewModel` queda atada al ciclo de vida del **NavGraph**. Esto significa que el ViewModel sobrevivirá intacto a la rotación de la pantalla (evitando la pérdida de datos y reinicios molestos) y se destruirá automáticamente solo cuando el usuario destruya la Actividad, previniendo así de forma efectiva cualquier fuga de memoria (memory leaks).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

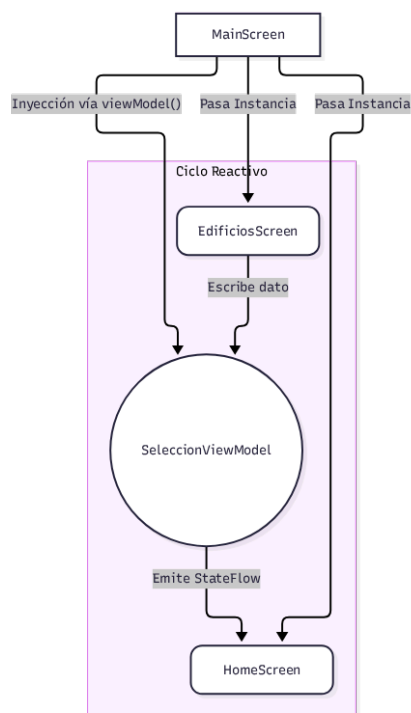


Figura 3: Diagrama de inyección y flujo reactivo del Shared ViewModel

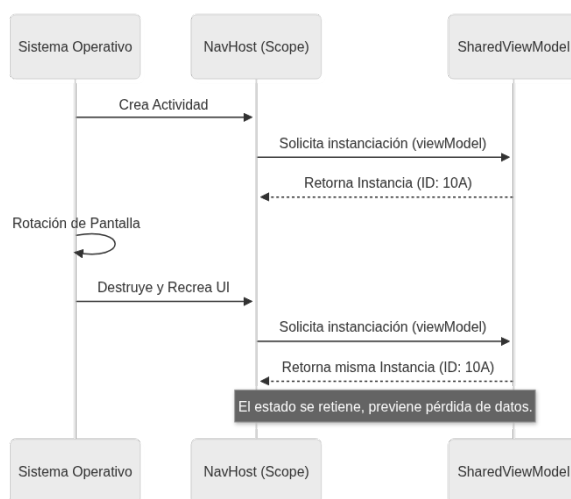


Figura 4: Secuencia del ciclo de vida del ViewModel atado al *scope* del NavHost

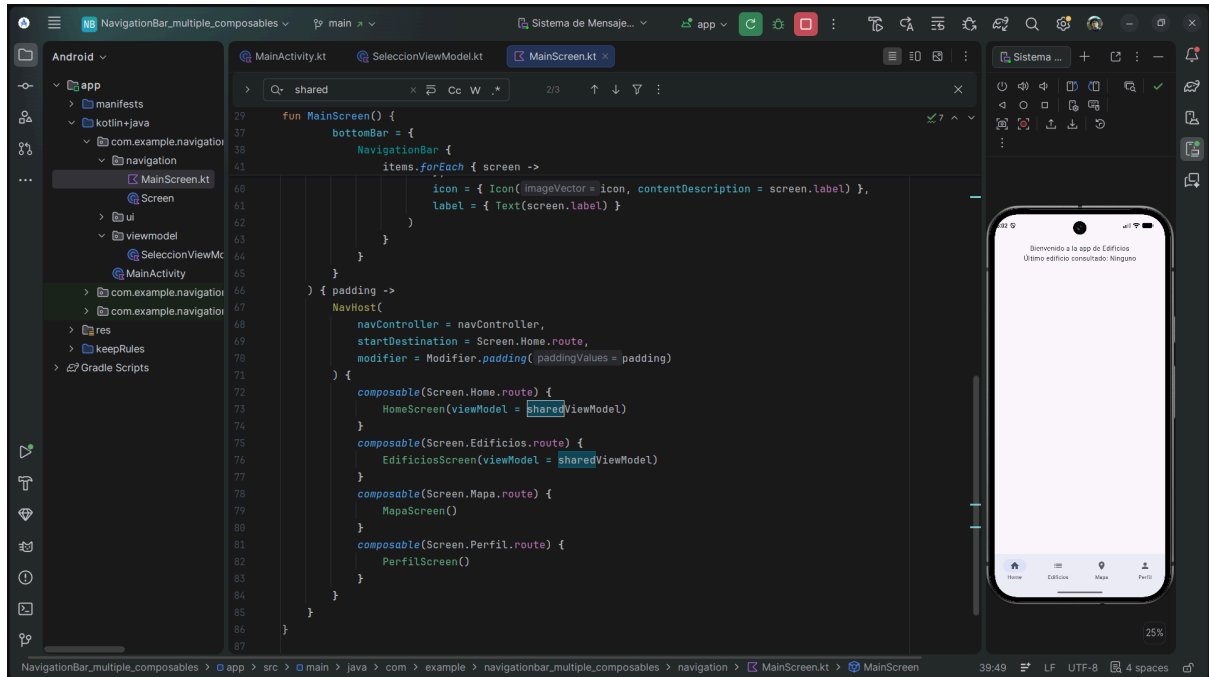


Figura 5: Inyección global del Shared ViewModel atado al ciclo de vida en MainScreen (Parte 1)

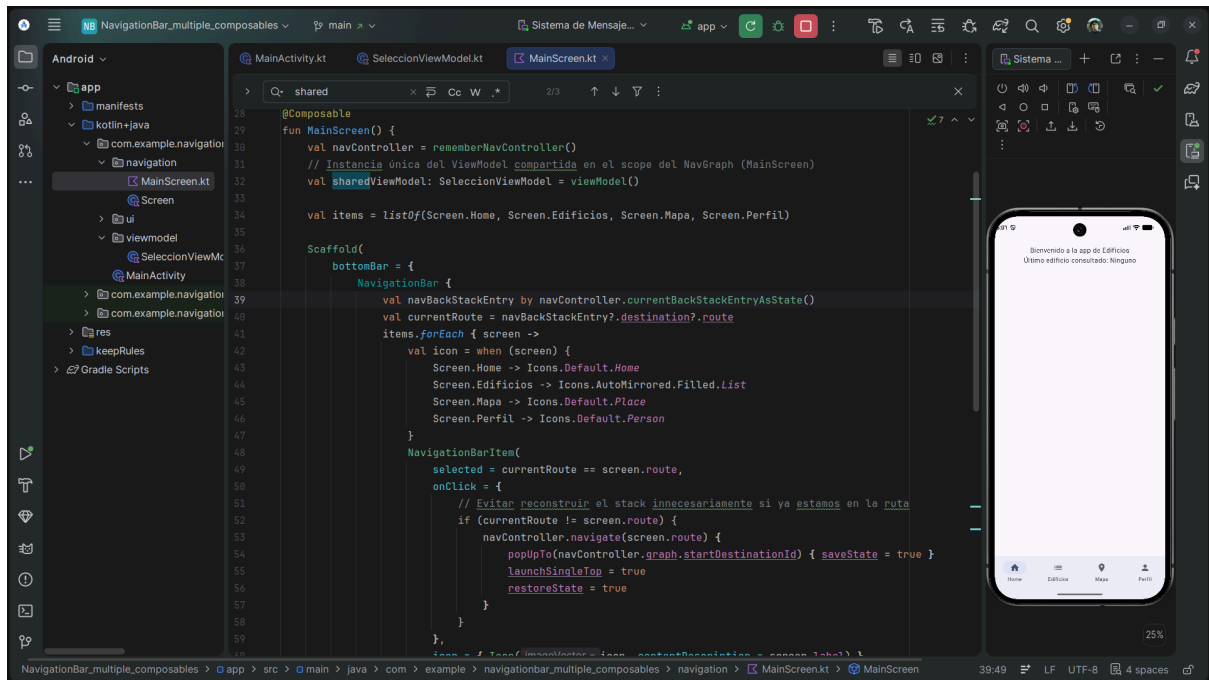


Figura 6: Inyección global del Shared ViewModel atado al ciclo de vida en MainScreen (Parte 2)

3. Delegación de Escritura en EdificiosScreen

A nivel de interfaz gráfica, el propósito de EdificiosScreen es simplemente pintar una lista de opciones. En la versión guiada por el docente, la vista enviaba el dato seleccionado llamando a la lambda `onEdificioSeleccionado`, la cual terminaba delegando la actualización hacia arriba.

En nuestra implementación mejorada, EdificiosScreen recibe la instancia inyectada de nuestro `ViewModel`. Al pulsar el botón “Ver” dentro de la iteración de la `LazyColumn`, se invoca la función pública `seleccionarEdificio(nombre)`. Esta importante decisión de arquitectura transfiere por completo la responsabilidad y la lógica de mutar estados desde la interfaz hacia el `ViewModel`. La UI pasa a ser un componente netamente declarativo y “tonto” (*dumb component*) que solo sabe renderizarse a sí mismo y avisarle al orquestador qué evento acaba de ocurrir.

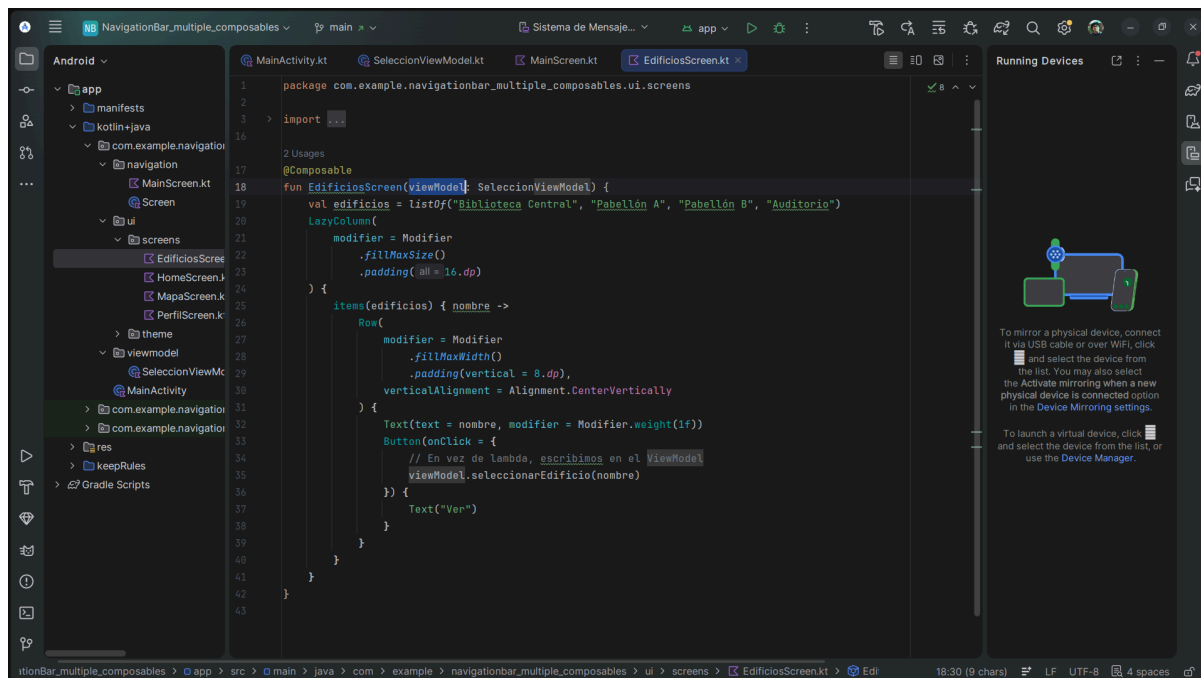


Figura 7: Transferencia de responsabilidad de escritura desde la UI hacia el orquestador

4. Recomposición Reactiva en HomeScreen

La verdadera fortaleza del paradigma declarativo y reactivo en Compose se aprecia en cómo HomeScreen lee la información. Para escuchar los cambios en tiempo real del edificio seleccionado, se empleó el operador `collectAsState()` sobre el flujo reactivo.

La justificación técnica detrás de esto es que `collectAsState()` suscribe internamente a nuestro Composable al flujo asíncrono emitido por el `ViewModel`. Cada vez que EdificiosScreen altera el valor en memoria, el flujo emite un nuevo `String`; el estado interno se actualiza e instruye directamente al compilador de Jetpack Compose a gatillar una **recomposición** inteligente. HomeScreen vuelve a ejecutarse redibujando el nuevo nombre del edificio sin que nosotros hayamos escrito ninguna lógica manual del tipo `textView.setText(...)`, resultando en un comportamiento predecible y un código mucho más robusto que lidiar con funciones lambda de retorno.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

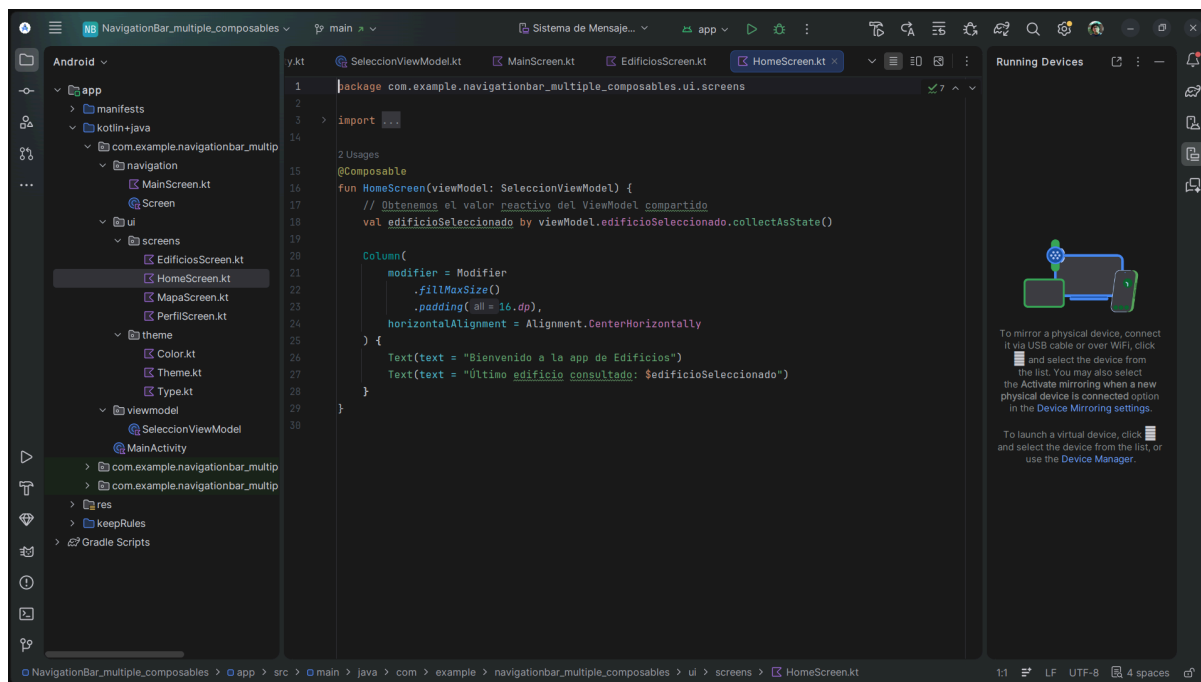


Figura 8: Suscripción reactiva y repintado de vista mediante `collectAsState` en `HomeScreen`

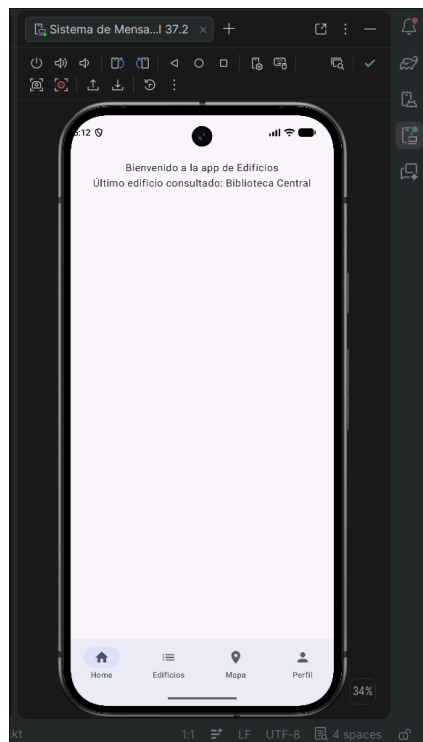


Figura 9: Pantalla `HomeScreen` reflejando de forma inmediata y automática el último cambio de estado dictado por la otra pestaña

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

5. Integración Fluida de la Cuarta Pestaña (Perfil)

Finalmente, el laboratorio solicitaba expandir las capacidades de enrutamiento agregando una cuarta sección a la barra de navegación de la aplicación. En el paradigma de *Navigation Compose* [3], este tipo de adiciones requieren intervenir exactamente tres puntos fuertemente estructurados por nuestro diseño:

1. **El enrutador tipado:** Se añadió el objeto `Perfil` a la clase sellada `Screen.kt`. Esto es sumamente útil y preferible a pasar simples *Strings* mágicos ("`perfil`"), ya que previene por completo los errores tipográficos fatales al intentar compilar rutas en el `NavHost`.
2. **El componente visual:** Se definió el archivo `PerfilScreen.kt` conteniendo un `Composable` básico alineado al centro que funge temporalmente como marcador de posición.
3. **El `NavigationBarItem`:** Se modificó la lista que iteraba la barra `bottomBar` para renderizar nuestro cuarto botón. Para esto, se reutilizó el diseño `Material Design` mediante el ícono pre-empaquetado `Icons.Default.Person`.

Una de las mayores ventajas de haber construido todo esto dentro de un contenedor `Scaffold` puro de `Compose` es que la barra inferior recalculó geométricamente su espacio y se distribuyó por sí sola para acomodar cuatro íconos y etiquetas, algo que con el antiguo sistema de `Views` y `XML` habríamos tenido que rediseñar a mano en los `layouts` de `Android`.

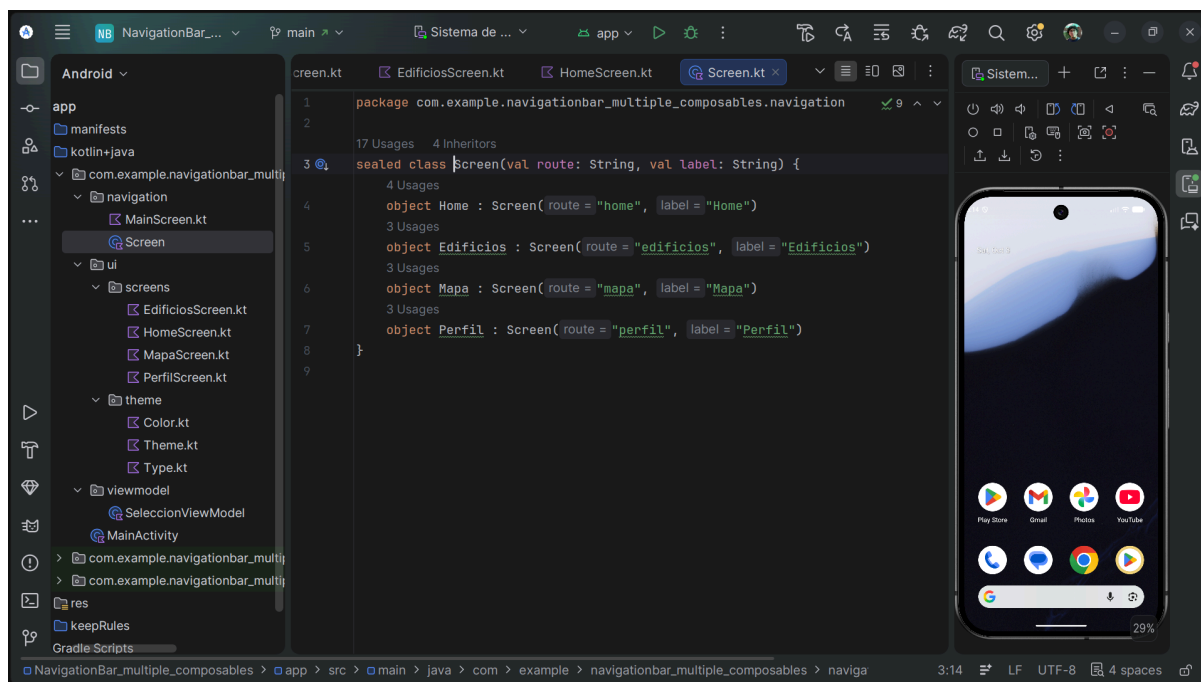


Figura 10: Definición estricta y segura de la nueva ruta en la clase sellada `Screen`

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

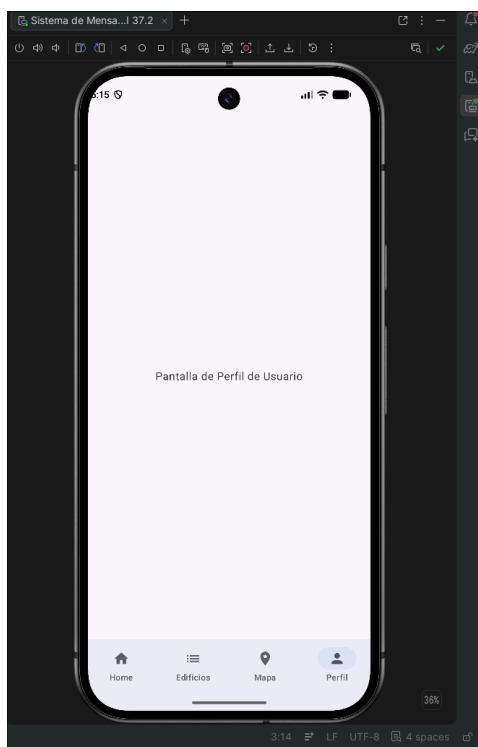


Figura 11: Visualización en tiempo real de la cuarta pestaña adaptándose fluidamente en el NavigationBar

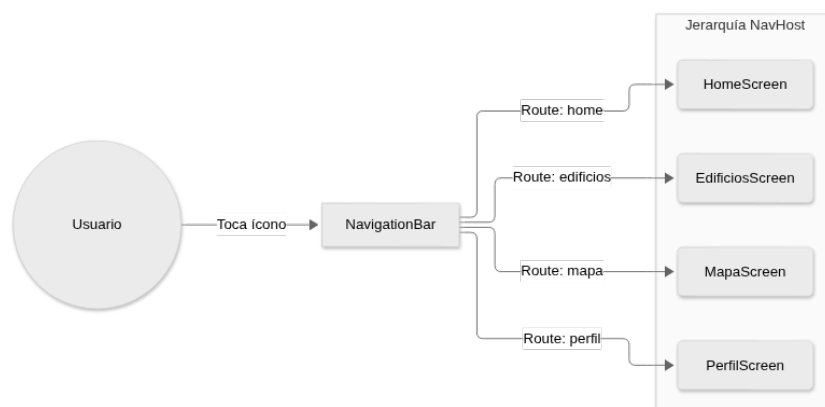


Figura 12: Esquema de enrutamiento y accesibilidad mediante el NavigationBar

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantallas Home, Edificios y Mapa navegables mediante NavigationBar, donde el edificio seleccionado en Edificios se refleja en Home)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, se logró implementar por completo la estructura solicitada de Navigation Compose. La comunicación fluye correctamente y el HomeScreen refleja la selección generada en EdificiosScreen.

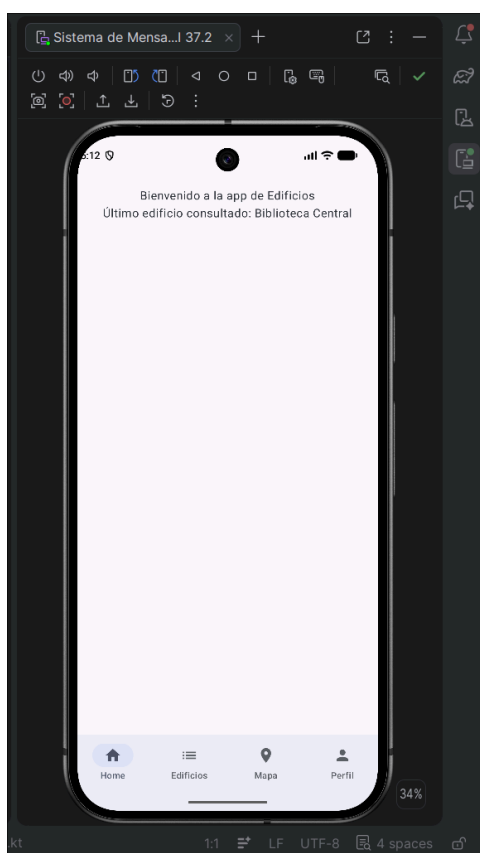


Figura 13: Pantalla Home reflejando el edificio seleccionado

2. En tu propia implementación, ¿qué diferencia encontraste entre comunicar Composables mediante una función lambda y hacerlo mediante un ViewModel compartido? ¿Cuál de las dos alternativas consideras una mejor práctica? Explica brevemente los motivos.

La principal diferencia es la **escalabilidad**. Con funciones lambda (State Hoisting básico), la información seleccionada debe “viajar” forzosamente como parámetro de vuelta hacia el Composable raíz (MainScreen) y luego descender hacia el otro Composable receptor. A medida que la app crece, este pasaje manual se vuelve insostenible (conocido como *prop drilling*). Por su parte, el **ViewModel compartido** proporciona un único punto de verdad (“Single Source of Truth”). Al inyectar la instancia de ViewModel a los Composables interesados, estos leen y escriben el estado de forma directa, reactiva y limpia. Definitivamente, el ViewModel es una mejor práctica para datos transversales o complejos dentro del scope de un flujo de navegación.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

3. En forma individual, explica qué ventaja adicional ofrece un ViewModel frente a otras formas de mantener el estado al navegar entre pantallas.

La ventaja más significativa del ViewModel es su **conciencia sobre el ciclo de vida**. A diferencia de una variable global (object singleton) o un simple remember, el ViewModel sobrevive a los cambios de configuración como las rotaciones de pantalla, pero se destruye automáticamente cuando el ciclo de vida asociado (por ejemplo, el NavGraph de la Activity actual) es destruido definitivamente. Esto impide de forma efectiva las fugas de memoria y reinicios molestos de datos al rotar un teléfono, todo ello sin tener que sobrecargar la pila de navegación intentando serializar objetos gigantes a través de los argumentos de ruta del NavController.

CONCLUSIONES

- **Eficiencia en el paradigma declarativo:** Se evidenció que la adopción de Jetpack Compose y Navigation Compose agiliza notablemente el desarrollo de flujos multi-pantalla y sus correspondientes NavigationBar, prescindiendo completamente de la complejidad y sobrecarga de ciclo de vida que tenían los antiguos Fragments.
- **Manejo maduro del estado:** Migrar de funciones lambda hacia una arquitectura con SharedViewModel permitió experimentar un patrón de diseño fundamental. El ViewModel actúa como orquestador del estado, centralizando la lógica de negocios y limpiando radicalmente la capa visual.
- **Integración sin fricciones:** Al utilizar utilidades de navegación como launchSingleTop y restoreState, se logra proveer al usuario de una navegación predecible sin apilar vistas duplicadas, resultando en un control de pila robusto y un comportamiento a la par de aplicaciones de clase mundial.

REFERENCIAS

Bibliography

- [1] Google Developers, "ViewModel Overview | Android Developers." Accessed: Oct. 03, 2024. [Online]. Available: <https://developer.android.com/topic/libraries/architecture/viewmodel>
- [2] JetBrains, "StateFlow and SharedFlow | Kotlin Documentation." Accessed: Oct. 03, 2024. [Online]. Available: <https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow/>
- [3] Google Developers, "Navigating with Compose | Android Developers." Accessed: Oct. 03, 2024. [Online]. Available: <https://developer.android.com/jetpack/compose/navigation>